

Reflection: Building a Multi-Agent Research Crew

UCLA Extension — Agentic AI | Week 1

Assignment Diego Carpenter

How I Designed the Agents and Why

When approaching this assignment, my main goal was to make sure each agent had a clear, distinct job so they wouldn't overlap or confuse each other. I ended up with four agents, each owning one stage of the research pipeline.

The **Query Expander** is purely a planning agent — no tools, just thinking. Its job is to take a broad research question and break it down into specific sub-questions and keywords. I gave it a backstory as an academic librarian because that felt like the right mental model: someone who knows how to think about a topic before searching for it. Without this step, the Source Hunter would just run one vague query and miss a lot of relevant material.

The **Source Hunter** is the only agent with a tool — the `search_papers` RAG tool. I kept tool access limited to just this agent intentionally. If every agent could search, the pipeline would get chaotic and it would be harder to track where information was coming from. The Source Hunter's entire job is to take the keyword clusters from the Query Expander and run focused searches, recording exactly where each finding came from.

The **Synthesizer** takes all those raw passages and finds the bigger picture — themes, agreements, disagreements, and gaps. This is where the output stops being a list of summaries and starts becoming actual analysis. I gave it a backstory as a senior researcher who has written survey papers, because that's exactly the skill needed here.

The **Report Writer** takes the synthesis and turns it into a properly structured literature review. I was strict in its instructions about citation format and about never including claims it couldn't trace back to a retrieved paper. This was important for keeping the output grounded in the actual corpus.

I used a sequential process because each agent genuinely depends on the previous one's output — there's no parallelism that makes sense here.

ReAct-Style Reasoning in the Agent Outputs

ReAct stands for Reason, Act, Observe, and then Reason again based on what you observed. I saw this most clearly in the Source Hunter agent during execution.

When the Source Hunter received the search plan, it didn't just run one search and call it done. It reasoned about which sub-question to address first, ran a search query, observed what came back, and then decided what to search next based on those results. For example, after searching for "agent architectures," it would look at what it got and then run a follow-up search with different keywords to fill in gaps. That loop — reason, search, look at results, reason again — is exactly the ReAct pattern in action.

The Query Expander also showed reasoning before acting, spending its entire turn thinking through the problem space before producing a structured plan. It

flagged ambiguities in the original question (like whether "agents" meant software agents, robots, or LLM-based agents) and made explicit scope decisions, which is a form of reasoning about uncertainty before committing to an action.

What Worked, What Didn't, and What I Would Change

Honestly, I was surprised by how well it worked. Having never built anything like this before, seeing the agents produce coherent, well-reasoned literature reviews was genuinely exciting. The outputs were structured, the citations were organized, and the thematic synthesis made sense.

That said, a few things could be improved. The Query Expander sometimes generated sub-questions that were a bit too general — things like "what role does NLP play in agents" — which meant the Source Hunter was searching for broad concepts rather than the specific papers in the corpus. The searches still returned good results, but more corpus-aware query expansion would help.

I would also add a verification step between the Source Hunter and the Synthesizer — a lightweight check that confirms at least a minimum number of distinct papers were retrieved before moving on. If the corpus search underperforms, the whole downstream output suffers, and right now there's no guard against that.

What Would Be Needed for Production

Running this system in a real production environment would require several additions beyond what was built here.

First, **cost controls** — each run makes many LLM calls and the costs can add up quickly at scale. Rate limiting and token budgets per agent would be necessary.

Second, **citation verification** — the report writer is instructed not to fabricate sources, but there's no automated check confirming that every citation in the final report actually appeared in the retrieved passages. A post-processing validation step would catch any hallucinated references.

Third, **error handling and retries** — if a tool call fails or an agent produces output that doesn't match the expected format, the whole pipeline currently breaks. Production systems need graceful degradation and retry logic.

Finally, **human-in-the-loop review** — for anything going out publicly, a human should review the final report before it's used. The agents do a good job, but they can miss nuance or misattribute ideas, and a quick human review catches those issues cheaply.

Conclusion

This assignment was a great introduction to multi-agent systems. The CrewAI framework made it straightforward to define agents with distinct roles and wire them into a pipeline. The most valuable insight was seeing how much the quality of downstream outputs depends on the quality of upstream agents — a weak query expansion leads to weak retrieval, which leads to a weaker report. Designing each agent carefully, with a clear role and specific instructions, turned out to be the most important part of the whole project.